An Interactive Pedagogical Framework for Divide and Conquer Algorithms

AN EDUCATIONAL WEB PORTAL — TEACHER & USER GUIDE

Author: Prof. Dana-Marciana Jebelean

ORCID: 0009-0003-3242-9055

Affiliation: "Ioan Jebelean" Theoretical High School, Romania

Live Platform URL: danajebelean.com/Edutainment/DEI

1. ABSTRACT

This educational resource provides a fully interactive, web-based platform designed to facilitate the understanding of Divide and Conquer algorithms. Aimed at high school students, competitive programming trainees, and computer science undergraduates, the portal translates abstract recursive concepts into intuitive, step-by-step visual simulations. The primary goal of this software is to bridge the gap between theoretical algorithm design and practical code implementation.

The platform adopts an "edutainment" approach, allowing users to trace code execution synchronously with visual data transformations. By providing granular control over the execution flow, it serves as an excellent Open Educational Resource (OER) for both traditional classroom teaching and self-paced, remote learning.

2. PEDAGOGICAL CONTEXT & LEARNING OBJECTIVES

Teaching recursion and the Divide and Conquer paradigm is notoriously difficult because it requires students to build a mental model of the call stack and track multiple states simultaneously. Traditional methods, such as dry-running code on a whiteboard, often fail to capture the dynamic nature of these algorithms. This portal was designed specifically to address these cognitive bottlenecks.

Upon utilizing this platform, students should be able to:

- Visually identify the base cases and the recursive steps within an algorithm.
- Understand the specific roles of the three main phases: *Divide* (breaking the problem), *Conquer* (solving the sub-problems recursively), and *Combine* (merging the solutions).

- Trace the execution of QuickSort and MergeSort algorithms, accurately predicting the movement of pointers and the partitioning of data.
- Evaluate and compare the memory constraints and time complexities of different sorting implementations.

3. DETAILED PLATFORM MODULES

The portal is divided into four main interactive modules, each meticulously designed to highlight specific characteristics of the Divide and Conquer methodology:

Module 01: Towers of Hanoi

Theoretical Concept: The quintessential introduction to recursion. It emphasizes the core principle of reducing a complex problem (moving N disks) into a simpler sub-problem (moving N-1 disks).

Simulation Features: Visual representation of the Disks and Rods. The platform highlights the active recursive function calls and provides a clear textual breakdown of the Divide, Conquer, and Combine steps as the animation unfolds, mapping directly to the mathematical recurrence relation.

Module 02: QuickSort Analysis

Theoretical Concept: In-place sorting and partitioning logic. It demonstrates an average time complexity of O(N log N) while utilizing minimal auxiliary memory.

Simulation Features: A highly detailed, step-by-step trace of the pivot selection. The module visually tracks the index movements (the `i` and `j` pointers) scanning the array from opposite ends. Students can observe in real-time how elements are swapped to ensure that everything to the left of the pivot is smaller, and everything to the right is larger.

Module 03: Merge Sort (Classic)

Theoretical Concept: Out-of-place sorting, emphasizing the "Combine" phase. It guarantees O(N log N) time complexity but introduces O(N) space complexity.

Simulation Features: True visualization of the array decomposition process. As the recursion unwinds, the platform displays the creation of the auxiliary `L` (Left) and `R` (Right) arrays. Users can see exactly how elements from these temporary arrays are compared one-by-one and reassembled back into the main array in sorted order.

Module 04: Optimized Merge Sort (Olympiad Level)

Theoretical Concept: Memory-efficient array merging tailored for competitive programming environments where execution time and memory limits are strictly enforced.

Simulation Features: Demonstrates advanced implementation techniques. Instead of dynamically allocating new sub-arrays at every recursive step (which is costly), this simulation shows how to use a single global auxiliary array, dramatically reducing overhead and maintaining peak execution speed.

4. TECHNICAL ARCHITECTURE & DEPLOYMENT

The entire suite was developed as a lightweight, client-side application utilizing standard Web Technologies (HTML5, CSS3, and Vanilla JavaScript). This architectural choice was intentional to ensure maximum accessibility for educational institutions.

Deployment Instructions: Extract the provided `.zip` archive. Because there is no reliance on server-side processing, databases, or external frameworks, no backend installation is required. The software can be run locally by simply opening the `index.html` file in any modern web browser (Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari). Furthermore, it can be easily hosted on basic school web servers or free hosting platforms.

5. CLASSROOM IMPLEMENTATION STRATEGIES

To maximize the educational impact of this software, teachers are encouraged to integrate it into active learning scenarios rather than using it merely for passive demonstration. Below

are recommended strategies:

- **The "Pause and Predict" Method:** While presenting on a smartboard, use the "Next Step" button to pause the simulation immediately before a crucial recursive call or data swap. Ask the classroom to predict the upcoming state: *"Where will the 'i' pointer move next?"* or *"What will be the exact contents of the Left array during this merge step?"*
- **Code Mapping:** Direct the students' attention to the synchronization between the visual animation and the corresponding C++ source code. Highlighting how a specific line of code translates into a visual change helps solidify algorithmic thinking.
- **Flipped Classroom:** Assign students to interact with a specific module at home before the lesson. Ask them to write down the sequence of pivot selections for a given array, then verify their manual calculations using the portal during class time.

6. ASSESSMENT & EVALUATION

The visualizations provided by this platform can directly inspire assessment materials. Teachers can generate static screenshots from various stages of the QuickSort or MergeSort simulations and include them in written examinations. Students can be asked to identify the current algorithmic step, determine the depth of the recursion tree at that exact moment, or write the snippet of code responsible for the depicted state.

This project and its associated files are distributed under the
Creative Commons Attribution 4.0 International (CC BY 4.0) license.
You are free to share and adapt the material for any purpose, provided appropriate credit is given.